

Binary Search Trees (Part III) - Tree Node Deletion

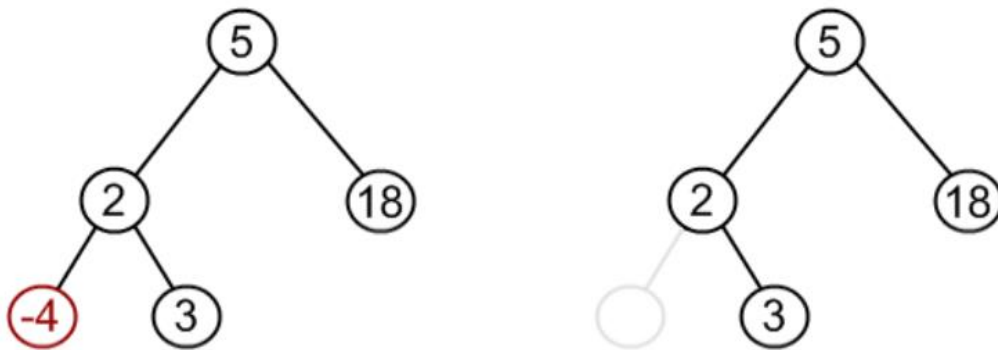
Purpose

- Understand and implement the algorithm for deleting a node from a binary search tree

Theory

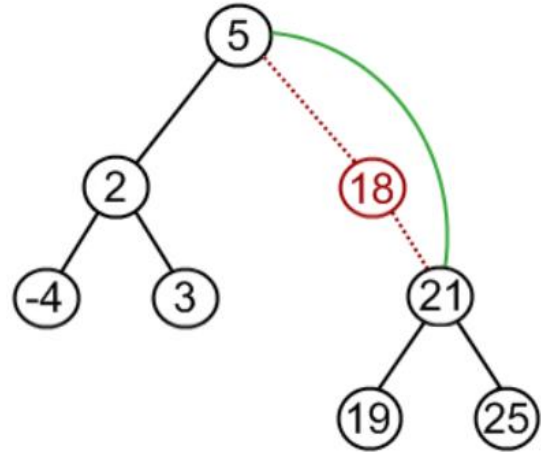
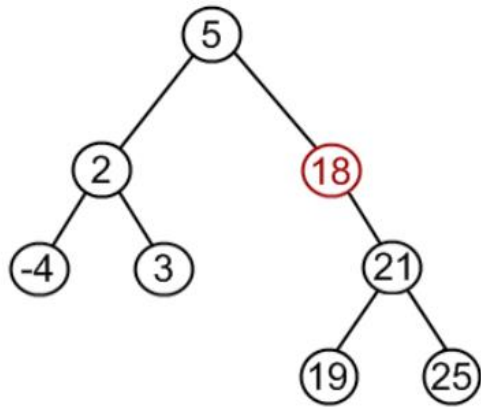
- Deletion of a node starts with a search for the node throughout the tree. The searching algorithm should be chosen according to the problem statement. (A reference to the found node's parent node is necessary in order to remove the node from the tree)
- If the node is found run the remove algorithm
- The remove algorithm should consider three main cases,
 - Node to be removed has no children
 - Has One child
 - Has both the children

1. No children for the node to be removed



➤ Simply remove the node from the tree by setting the reference of the parent node to null.

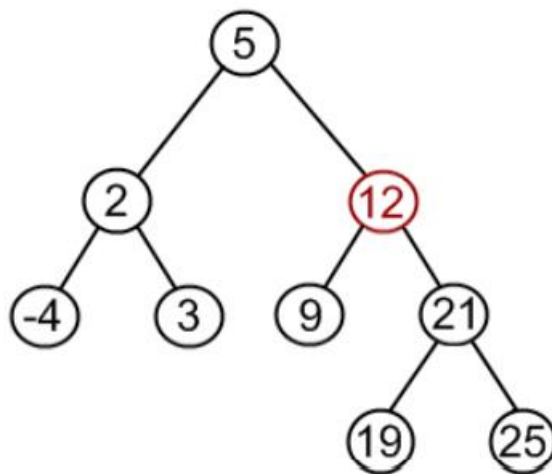
2. Removing Node has one child



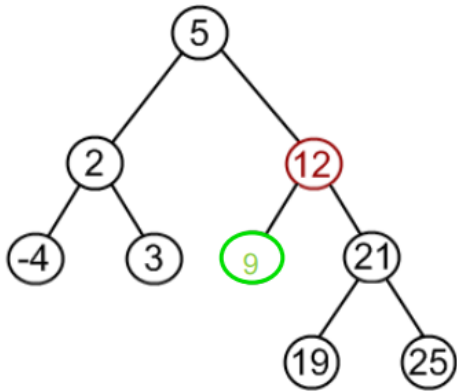
- Remove the node from the tree
- Joint the single child (with its subtree) to the parent of the removing node replacing the removing node.

3. Node to remove has two children

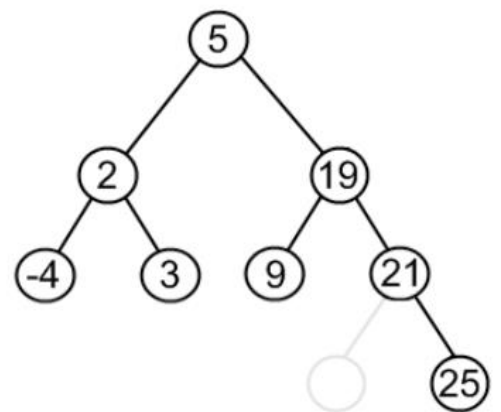
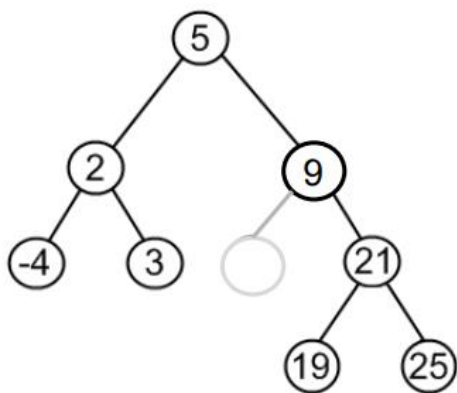
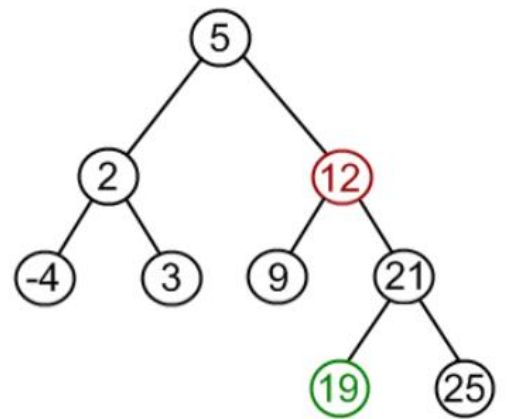
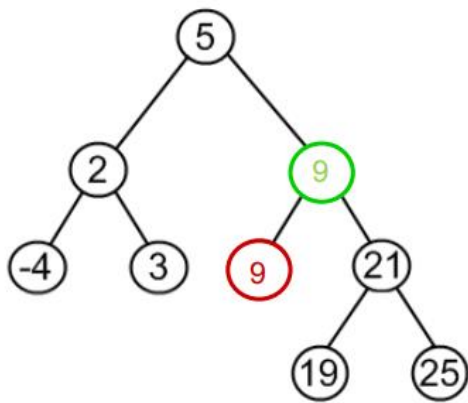
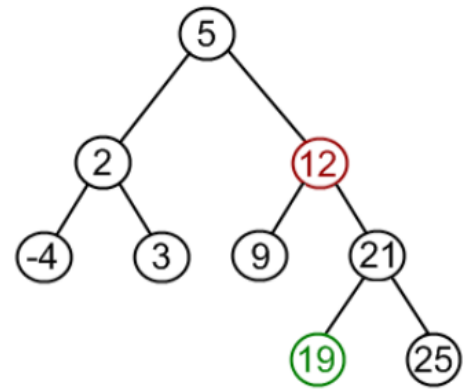
- Find the minimum value of the right sub-tree (leftmost leaf node of the right sub-tree) or the maximum value of the left sub-tree (rightmost leaf node of the left sub-tree).
- Replace the data part of the found node (minimum node in the right sub-tree or maximum value of the left sub tree) with the data part of the data to be removed.
- Apply the remove algorithm to the particular sub-tree in order to remove the found node. (Notice that since it is a leaf node and applying remove algorithm may result in 1st case or second case instance)



Maximum of left sub-tree

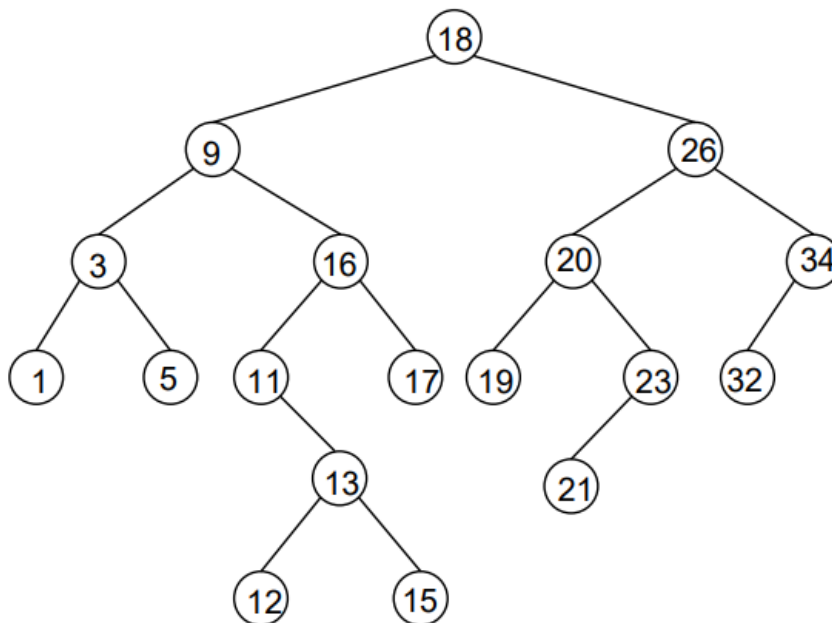


Minimum of right sub-tree



Question 01

1. Implement the binary search tree displayed below and create the following methods
 - a. Three traversal methods (inorder, preorder, postorder)
 - b. A method to search and return a reference to the searched node which searches nodes closer to the root node first.
 - c. A method to delete a particular node that uses the maximum of the left sub-tree in order to replace the deleted node.
 - d. A method to delete a particular node that uses a minimum of right sub-tree in order to replace the deleted node.



- e. Execute the following and print the resulting tree in the three traversals after each part,
 - i. Delete node with value 1.
 - ii. Delete node with value 34.
 - iii. Delete node with value 9 using minimum of right subtree method.
 - iv. Delete node with value 20 using maximum of left subtree method.